UNIVERSIDAD DE LA CUENCA DEL PLATA

Facultad de Ingeniería, Tecnología y Arquitectura

Carrera: Ingeniería en Sistemas de Información

Actividad #8: Integración Continua aplicada al proyecto EmpreAssist

Documentación técnica en formato APA

Materia: Ingeniería de Software

Proyecto TIF: EmpreAssist

Módulo funcional seleccionado: Stock y pedidos a proveedores con asistencia de IA

Integrantes:

Usui José

Waniukiewicz Nicolás

Stupniki Hernán

Tarnowski Tobias

Año: 2026

Comisión: A

Resumen

El presente documento desarrolla la Actividad #8 de Integración Continua aplicada al proyecto EmpreAssist, una plataforma orientada a dueños de pequeños negocios que necesitan organizar stock, pedidos, ventas, tareas y personal de manera simple. Para cumplir con la consigna, se seleccionó el módulo funcional de stock y pedidos a proveedores, ya que representa una necesidad central del usuario principal del sistema: evitar faltantes, ordenar la reposición y reducir errores operativos. La actividad propone una estrategia de gestión de configuración basada en Git, ramas de trabajo por integrante, control de cambios mediante pull requests y automatización de calidad mediante GitHub Actions. El pipeline configurado ejecuta instalación de dependencias, verificación de estilo con linter, pruebas unitarias y compilación del proyecto. Además, se documentan errores simulados y su resolución, con el objetivo de evidenciar cómo la integración continua permite detectar fallas tempranamente antes de integrar cambios a la rama principal.

Palabras clave: integración continua, gestión de la configuración, GitHub Actions, pruebas unitarias, linter, EmpreAssist.

1. Introducción

La integración continua es una práctica de desarrollo de software que permite integrar cambios de forma frecuente, verificando automáticamente que el producto mantenga condiciones mínimas de calidad. En el marco de la Gestión de la Configuración, esta práctica ayuda a identificar versiones, controlar modificaciones, detectar errores y sostener la trazabilidad entre requisitos, código, pruebas y entregables.

En la Actividad #7 se trabajaron conceptos vinculados con la identificación de elementos de configuración, control de versiones, trazabilidad, construcción del producto y automatización del proceso. A partir de esa base conceptual, esta actividad aplica esos principios sobre un módulo concreto del Trabajo Integrador Final: EmpreAssist.

2. Contexto del proyecto EmpreAssist

EmpreAssist es una plataforma digital orientada a pequeños negocios y comercios que venden productos. Su objetivo es simplificar la gestión diaria del negocio mediante una herramienta clara, modular y adaptable. El proyecto responde a una problemática frecuente: muchos dueños de negocios registran información en papel, memoria, WhatsApp o planillas simples, lo que genera desorden, errores y pérdida de tiempo.

El público objetivo principal está compuesto por dueños de pequeñas empresas o locales comerciales, como kioscos, despensas, locales de comida, tiendas de ropa y comercios similares. El público secundario está formado por encargados o empleados con permisos limitados, quienes pueden colaborar cargando stock, registrando novedades o actualizando información operativa.

Entre las funcionalidades previstas se encuentran la gestión de stock, pedidos a proveedores, registro de ventas, asistencia de empleados, recordatorios y tareas operativas. Por este motivo, el módulo de stock y pedidos resulta adecuado para aplicar integración continua, ya que concentra lógica de negocio crítica y puede verificarse mediante pruebas automatizadas.

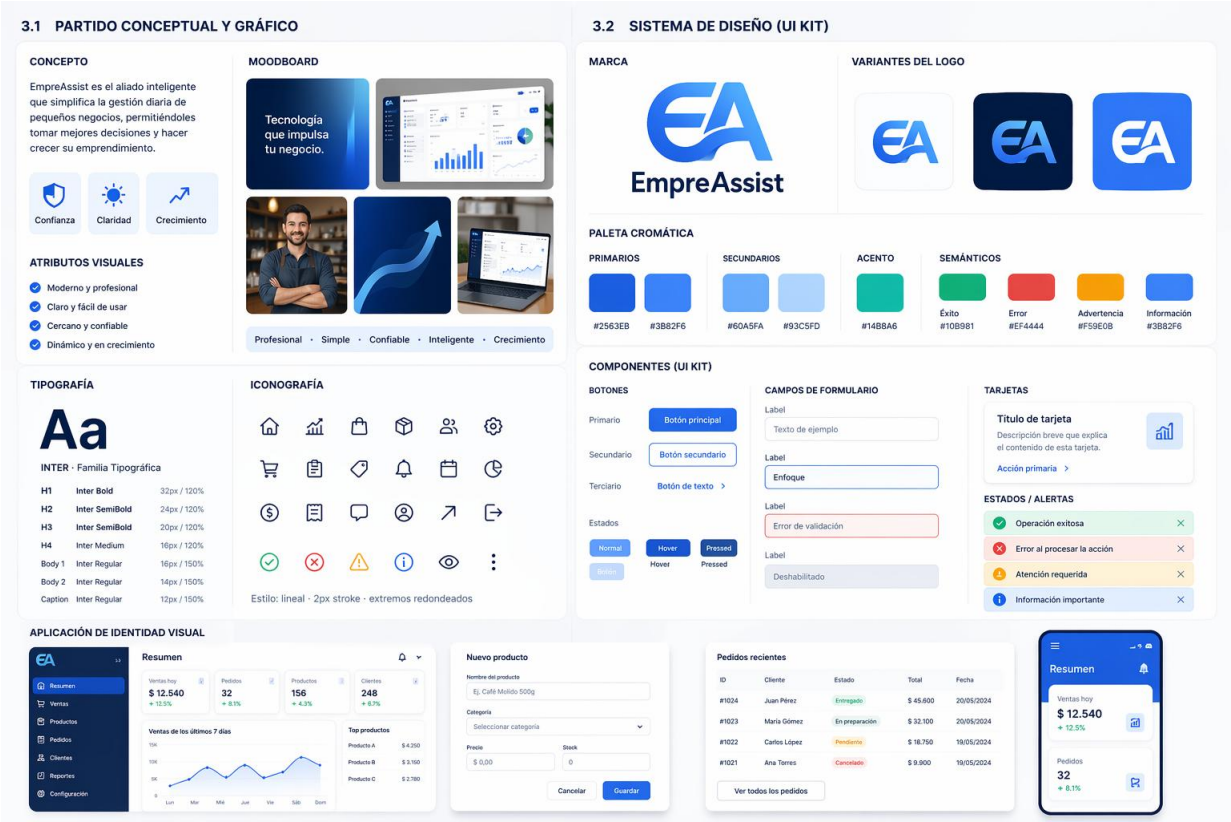

Figura 1. Sistema de diseño e identidad visual de EmpreAssist.

3. Módulo funcional seleccionado

El módulo seleccionado para esta actividad es "Stock y pedidos a proveedores con asistencia de IA". Este módulo permite controlar productos, detectar faltantes, generar alertas de bajo stock y preparar pedidos a proveedores. Se lo eligió porque responde directamente a los dolores del usuario principal: falta de control, dependencia de la memoria, pedidos tardíos y necesidad de ordenar la información del negocio.

Elemento	**Descripción**
Nombre del módulo	Stock y pedidos a proveedores
Usuario principal	Dueño del negocio
Usuario secundario	Encargado o empleado con permisos limitados
Necesidad que resuelve	Controlar productos, detectar faltantes y organizar pedidos

Valor para el proyecto	Reduce errores operativos y mejora la toma de decisiones
Tipo de pruebas posibles	Validaciones de stock, permisos, cálculos de pedido y alertas

4. Aplicación de Gestión de la Configuración

Según el enfoque trabajado en la actividad anterior, el proyecto requiere identificar los elementos de configuración y controlarlos mediante una estrategia clara. En EmpreAssist, los principales elementos de configuración son el código fuente, los archivos de configuración, las pruebas unitarias, el pipeline de CI, la documentación técnica y las evidencias de ejecución.

Elemento de configuración	**Ejemplo en el proyecto**	**Control aplicado**
Código fuente	src/lib/stock.ts	Versionado en Git y revisión por pull request
Pruebas unitarias	src/lib/stock.test.ts	Ejecución automática en el pipeline
Configuración de CI	.github/workflows/ci.yml	Control de cambios y validación en cada push/PR
Dependencias	package.json y package-lock.json	Instalación reproducible con npm ci
Documentación	README.md y documento APA	Actualización vinculada al avance del módulo
Backlog	Tarjeta de integración continua en la herramienta TIF	Trazabilidad entre tarea, código y evidencias

5. Organización del repositorio

El repositorio se organiza con una rama principal estable, una rama de integración y ramas de desarrollo por integrante. Esta estructura permite dividir responsabilidades, evitar modificaciones directas sobre main y revisar los cambios antes de integrarlos.

Rama	**Responsable**	**Uso**
main	Equipo completo	Versión estable del proyecto

develop	Equipo completo	Integración previa a main
dev/usui-stock	Usui José	Lógica del módulo de stock
dev/stupniki-tests	Stupniki Hernán	Pruebas unitarias
dev/tarnowski-ui	Tarnowski Tobias	Interfaz y componentes visuales
dev/waniukiewicz-ci	Waniukiewicz Nicolás	Pipeline, linter y documentación técnica

La política de integración propuesta es la siguiente: ningún cambio se integra directamente a main; cada integrante trabaja en su rama; los cambios se proponen mediante pull request; el pipeline debe finalizar correctamente antes de fusionar; y cada error detectado debe resolverse antes de aceptar la integración.

6. Inclusión de la tarea en el backlog del TIF

La consigna solicita incluir esta tarea dentro del backlog del producto del Trabajo Integrador Final. Para ello se propone la siguiente historia de usuario:

Campo	**Contenido**
Historia de usuario	Como equipo de desarrollo, queremos configurar integración continua para el módulo de stock y pedidos, para detectar automáticamente errores de compilación, pruebas y estilo antes de integrar cambios a la rama principal.
Prioridad	Alta
Estimación	5 puntos
Responsables	Equipo completo
Criterios de aceptación	1) Existe repositorio Git con main y ramas de desarrollo. 2) Existe archivo ci.yml. 3) El pipeline ejecuta lint, tests y build. 4) Hay al menos 5 pruebas unitarias. 5) Se documentan errores simulados y resolución.
Definición de terminado	La tarea se considera terminada cuando el pipeline se ejecuta correctamente y el README/documento técnico contienen evidencias y explicación del proceso.

6.1 Enlaces del proyecto

Repositorio GitHub:

https://github.com/nicolaswaniukiewicz/empreassist-ci

Archivo de configuración del pipeline:

https://github.com/nicolaswaniukiewicz/empreassist-ci/blob/main/.github/workflows/ci.yml

Pipeline CI:

https://github.com/nicolaswaniukiewicz/empreassist-ci/actions

Herramienta de gestión del TIF / Backlog:

https://trello.com/b/RhDKbxrE/empreassist

7. Herramienta de Integración Continua seleccionada

Se seleccionó GitHub Actions porque se integra directamente con repositorios GitHub, permite ejecutar flujos automáticos ante eventos como push o pull request, y resulta suficiente para validar el módulo en esta etapa del proyecto. La herramienta elegida automatiza pasos que antes deberían hacerse manualmente, reduciendo errores humanos y mejorando la calidad del proceso de integración.

Archivo de configuración del pipeline:

```
name: CI - EmpreAssist


on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]


jobs:
  build-test-lint:
    name: Build, tests and lint
    runs-on: ubuntu-latest
```

```yaml
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4


      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm


      - name: Install dependencies
        run: npm ci


      - name: Run linter
        run: npm run lint


      - name: Run unit tests
        run: npm test


      - name: Build project
        run: npm run build
```

8. Pasos automatizados del pipeline

1. Checkout repository: descarga el código fuente del repositorio para que el runner pueda trabajar sobre la última versión.
2. Setup Node.js: configura el entorno de ejecución con Node.js 20.
3. Install dependencies: instala dependencias usando npm ci, lo cual favorece una instalación reproducible a partir del archivo lock.
4. Run linter: verifica estándares de estilo y posibles errores de calidad estática.
5. Run unit tests: ejecuta las pruebas automatizadas del módulo seleccionado.
6. Build project: compila o verifica el proyecto para comprobar que no existan errores de TypeScript o construcción.
7. Status report: GitHub Actions muestra el resultado del flujo como exitoso o fallido.

8.1 Evidencia de ejecución exitosa del pipeline

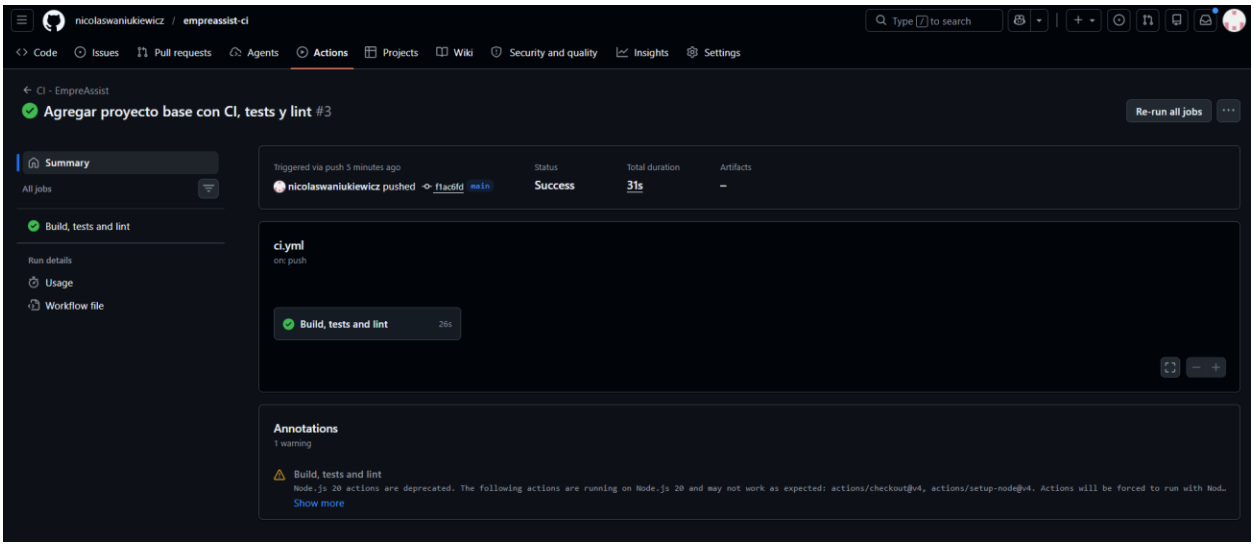

Figura 2. Ejecución exitosa del pipeline de integración continua en GitHub Actions. Se observa que el flujo CI - EmpreAssist finalizó correctamente sobre la rama main.

9. Pruebas unitarias implementadas

Para la actividad se definió un conjunto mínimo de pruebas unitarias sobre funciones puras del módulo de stock. Estas pruebas permiten validar reglas de negocio sin depender todavía de una interfaz final o de una base de datos.

N.º	Prueba	Resultado esperado
1	Validar producto con datos correctos	El producto es aceptado
2	Rechazar producto con precio negativo	La validación retorna falso
3	Detectar bajo stock	El sistema identifica stock actual menor o igual al mínimo
4	Calcular total de pedido	El total coincide con la suma de cantidad por precio unitario
5	Verificar permiso de encargado	El encargado puede cargar stock
6	Generar mensaje de reposición	El mensaje incluye productos faltantes

Código base de las funciones evaluadas:

```
export function validarProducto(producto: Producto): boolean {
  return (
    producto.id.trim().length > 0 &&
    producto.nombre.trim().length > 0 &&
    producto.precio >= 0 &&
    producto.stockActual >= 0 &&
    producto.stockMinimo >= 0
  );
}


export function tieneBajoStock(producto: Producto): boolean {
  return producto.stockActual <= producto.stockMinimo;
}


export function calcularTotalPedido(items: ItemPedido[]): number {
  return items.reduce((total, item) => total + item.cantidad * item.precioUnitario, 0);
}


export function puedeCargarStock(rol: RolUsuario): boolean {
  return rol === 'duenio' || rol === 'encargado';
}
```

9.1 Evidencia de ejecución local de pruebas unitarias

```
C:\Users\usuario\Desktop\empreassist_actividad8_entregables\empreassist_actividad8>npm test

> empreassist-stock-ci@1.0.0 test
> vitest run


 RUN  v2.1.9 C:/Users/usuario/Desktop/empreassist_actividad8_entregables/empreassist_actividad8

 ✓ src/lib/stock.test.ts (6)
   ✓ Modulo de stock de EmpreAssist (6)
     ✓ valida un producto con datos correctos
     ✓ rechaza productos con precio negativo
     ✓ detecta bajo stock cuando el stock actual es menor o igual al minimo
     ✓ calcula correctamente el total de un pedido
     ✓ permite que un encargado cargue stock
     ✓ genera un mensaje de reposicion con productos faltantes

 Test Files  1 passed (1)
      Tests  6 passed (6)
   Start at  20:25:58
   Duration  1.16s (transform 664ms, setup 0ms, collect 649ms, tests 4ms, environment 0ms, prepare 178ms)
```

Figura 3. Ejecución local de las pruebas unitarias del módulo de stock. Se observa que las 6 pruebas definidas fueron ejecutadas correctamente mediante Vitest.

10. Verificación de estándares de estilo

El linter permite detectar errores de estilo, variables no utilizadas, uso incorrecto de tipos o patrones que pueden afectar la mantenibilidad del código. En este proyecto se propone ESLint para revisar archivos TypeScript antes de integrar cambios. Si el linter detecta una infracción, el pipeline falla y el equipo debe corregir el problema antes de fusionar el pull request.

11. Evidencias del proceso de integración continua

A continuación se presentan evidencias reales del proceso de configuración, ejecución y validación de la integración continua en el repositorio GitHub del proyecto EmpreAssist.

11.1 Repositorio GitHub y estructura del proyecto

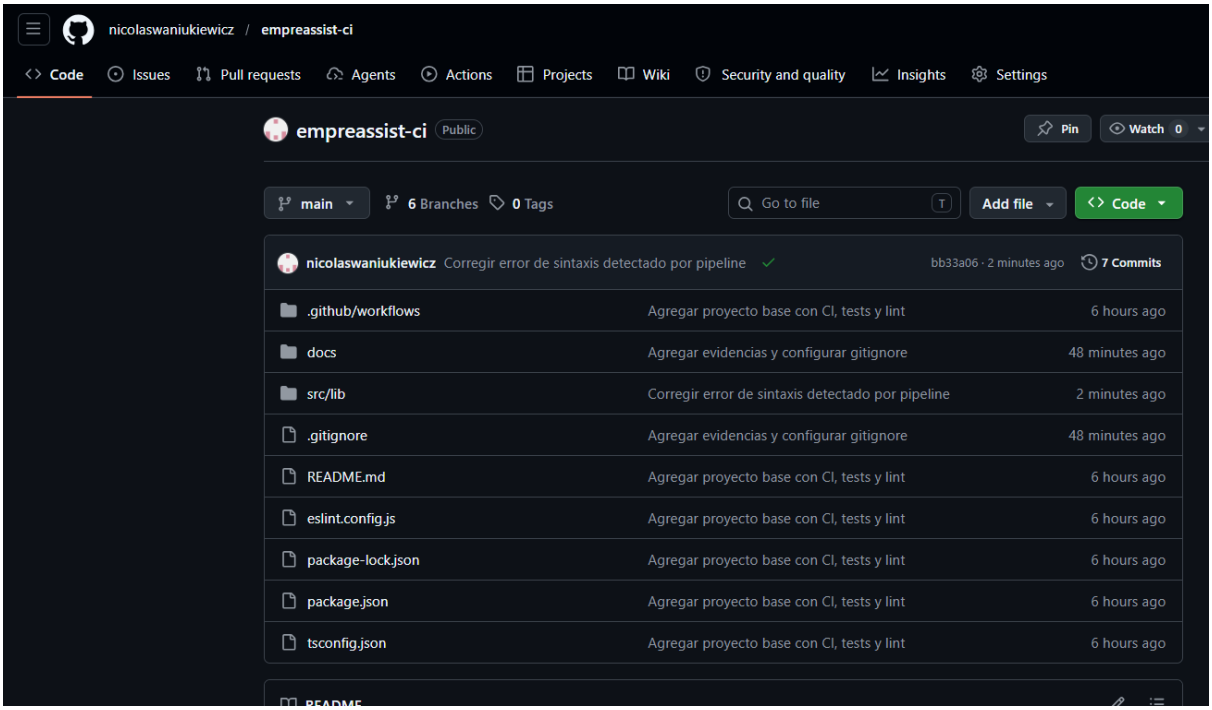

Figura 4. Repositorio GitHub del proyecto EmpreAssist con la estructura principal del proyecto, incluyendo código fuente, documentación, configuración del pipeline y archivos de dependencias.

11.2 Archivo de configuración del pipeline

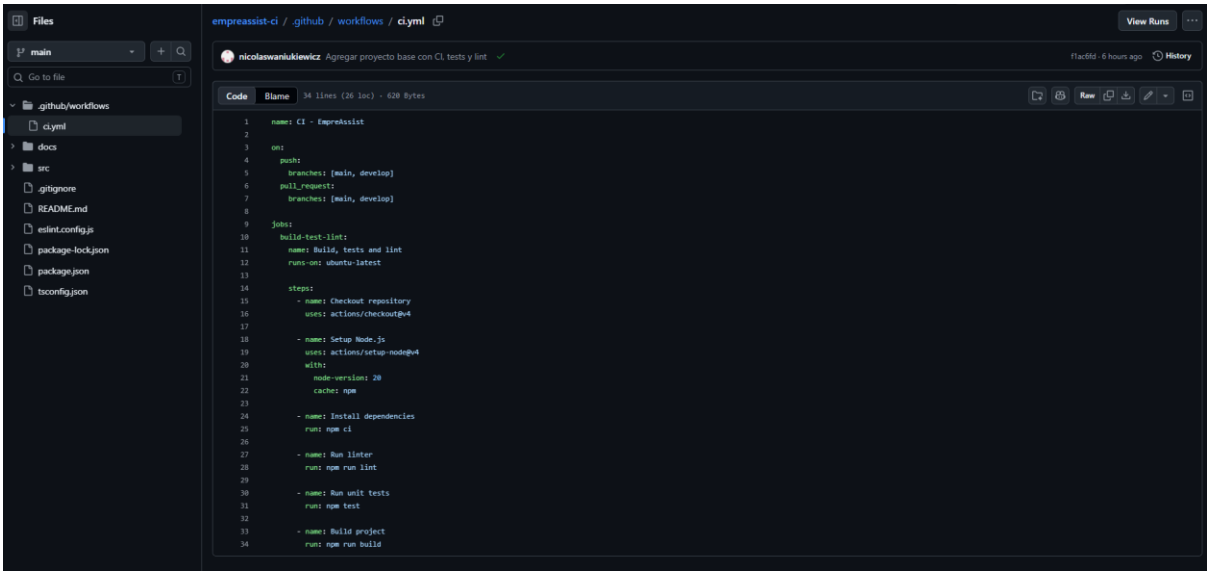

Figura 5. Archivo de configuración ci.yml ubicado en .github/workflows, utilizado por GitHub Actions para ejecutar automáticamente el pipeline de integración continua.

11.3 Ramas del repositorio

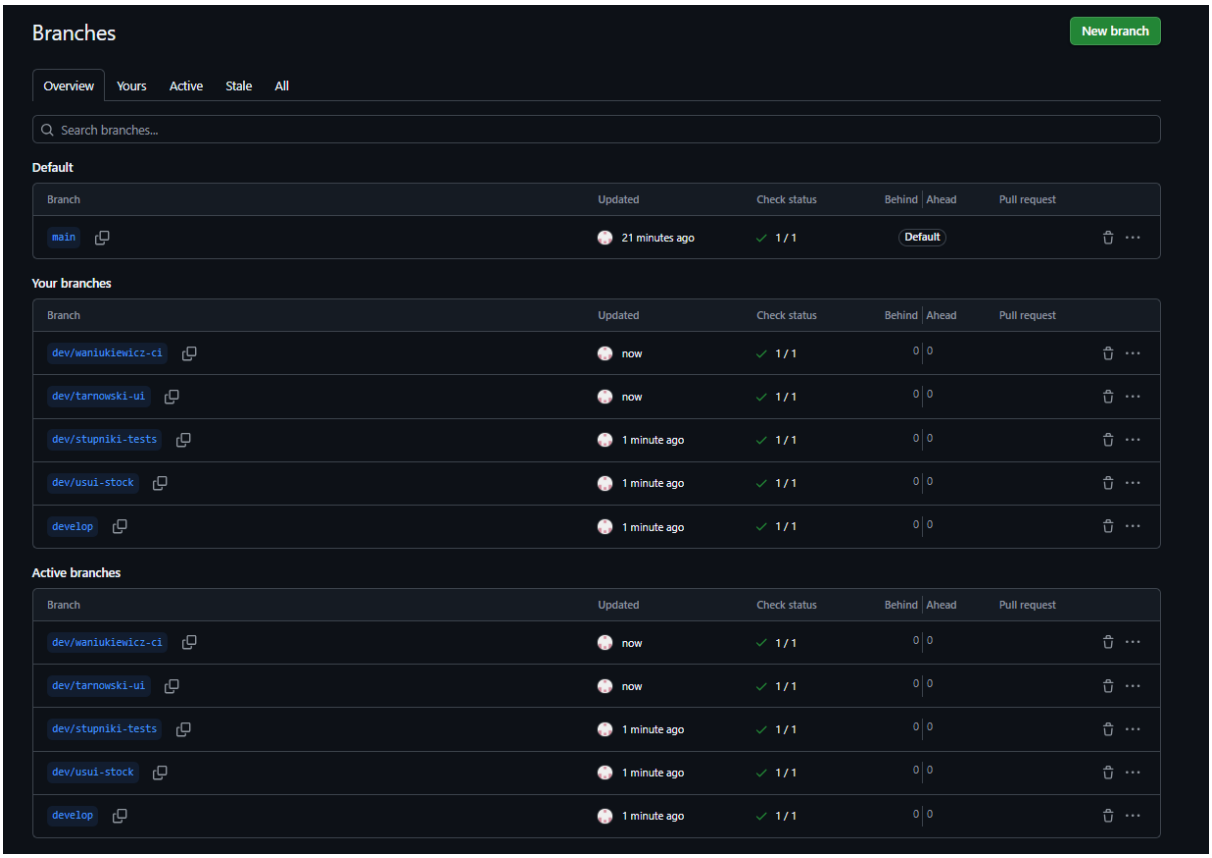

Figura 6. Ramas del repositorio utilizadas para organizar el trabajo del equipo: main, develop y ramas de desarrollo por integrante.

11.4 Tarea incluida en el backlog del TIF

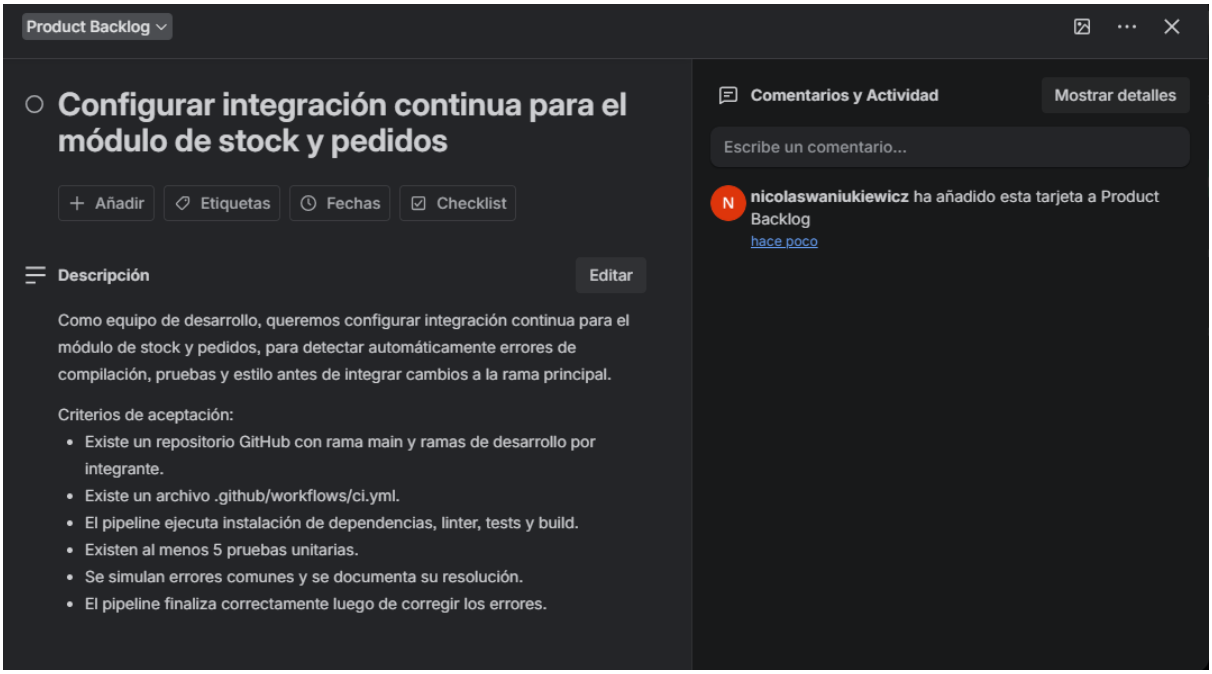

Figura 7. Tarjeta del backlog del TIF donde se registra la tarea de configuración de integración continua para el módulo de stock y pedidos.

12. Simulación de conflictos y errores comunes

La consigna solicita simular al menos dos conflictos o errores comunes. En esta propuesta se documentan tres casos para evidenciar el aporte de la integración continua.

Caso	Error simulado	Detección	Resolución
1	Un test falla tras una fusión de ramas porque se modificó la regla de bajo stock.	GitHub Actions marca como fallido el paso Run unit tests.	Se revisa la regla de negocio y se corrige la función tieneBajoStock para que use <= según el criterio definido.

2	El linter detecta una variable declarada y no utilizada dentro del módulo de stock.	GitHub Actions marca como fallido el paso Run linter.	Se elimina la variable o se utiliza correctamente. Luego se vuelve a ejecutar el pipeline.
3	Problema de dependencias: falta una dependencia de desarrollo para ejecutar los tests.	El pipeline falla durante npm ci o npm test.	Se agrega la dependencia correspondiente al package.json y se actualiza el lockfile.

12.1 Error simulado: prueba unitaria fallida

Para comprobar el funcionamiento del pipeline ante fallos de pruebas, se modificó temporalmente un valor esperado dentro del archivo stock.test.ts. Esta modificación provocó que una prueba unitaria fallara durante la etapa Run unit tests del pipeline.

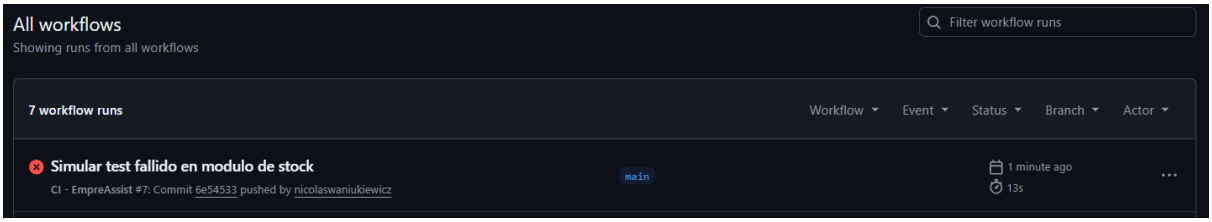

Figura 8. Ejecución fallida del pipeline provocada por una prueba unitaria incorrecta en el módulo de stock.

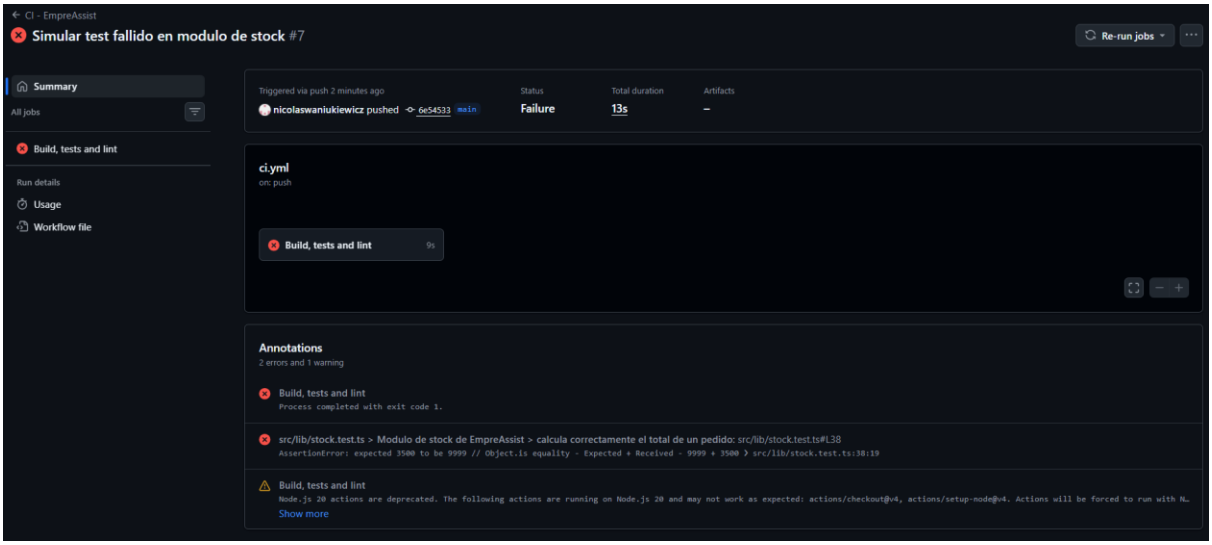

Figura 9. Detalle del error detectado durante la ejecución de las pruebas unitarias.

Luego se corrigió el valor esperado de la prueba y se volvió a ejecutar el pipeline, obteniendo un resultado exitoso.

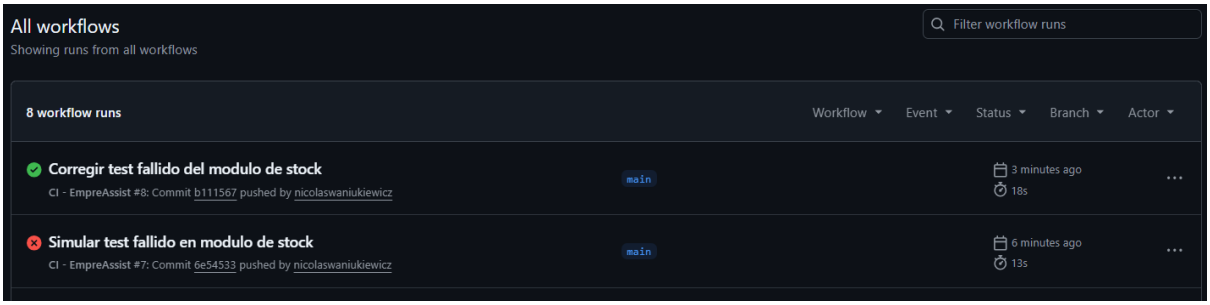

Figura 10. Pipeline ejecutado correctamente luego de corregir la prueba unitaria fallida.

12.2 Error simulado: error de sintaxis o estilo detectado por el pipeline

Para simular un segundo error común, se incorporó temporalmente una línea incompleta en el archivo stock.ts. Esto permitió comprobar que el pipeline detecta problemas de sintaxis o calidad antes de aceptar cambios en el repositorio.

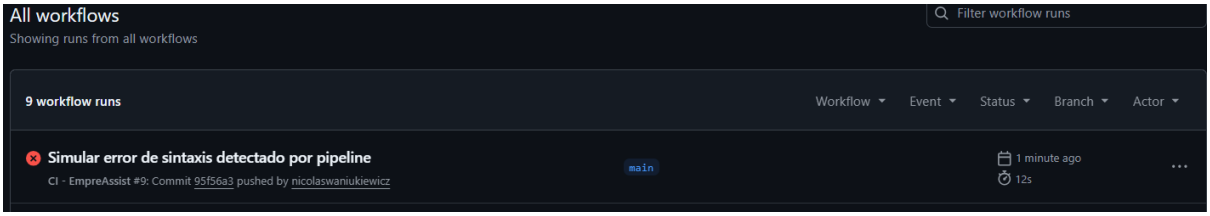

Figura 11. Ejecución fallida del pipeline debido a un error de sintaxis o estilo detectado durante la validación automática.

Luego se eliminó la línea incorrecta y se volvió a ejecutar el pipeline, obteniendo nuevamente un estado exitoso.

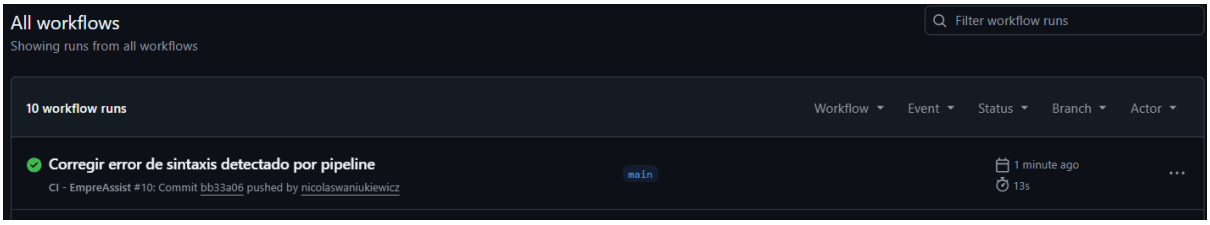

Figura 12. Ejecución final exitosa del pipeline luego de corregir los errores simulados.

Ejemplo de error de test:

```
FAIL src/lib/stock.test.ts
  Modulo de stock de EmpreAssist
    × detecta bajo stock cuando el stock actual es menor o igual al minimo


Expected: true
Received: false


Causa: la función usaba producto.stockActual < producto.stockMinimo.
Corrección: se reemplazó por producto.stockActual <= producto.stockMinimo.
```

Ejemplo de error de linter:

```
src/lib/stock.ts
  12:7  warning  'stockTemporal' is assigned a value but never used  no-unused-vars


Causa: quedó una variable temporal luego de una prueba manual.
Corrección: se eliminó la variable no utilizada.
```

13. Relación con la trazabilidad del proyecto

La actividad permite relacionar el backlog, el repositorio, los commits, los pull requests y las ejecuciones del pipeline. Esta relación aporta trazabilidad porque cada cambio puede vincularse

con una necesidad del producto y con una evidencia de validación. Por ejemplo, la historia de usuario de integración continua queda conectada con el archivo ci.yml, los tests del módulo de stock, las capturas de ejecución y el README del proyecto.

14. README.md solicitado

El archivo README.md fue incorporado al repositorio y contiene las instrucciones para clonar el proyecto, instalar dependencias, ejecutar pruebas, correr el linter y comprender el funcionamiento general del pipeline de integración continua.

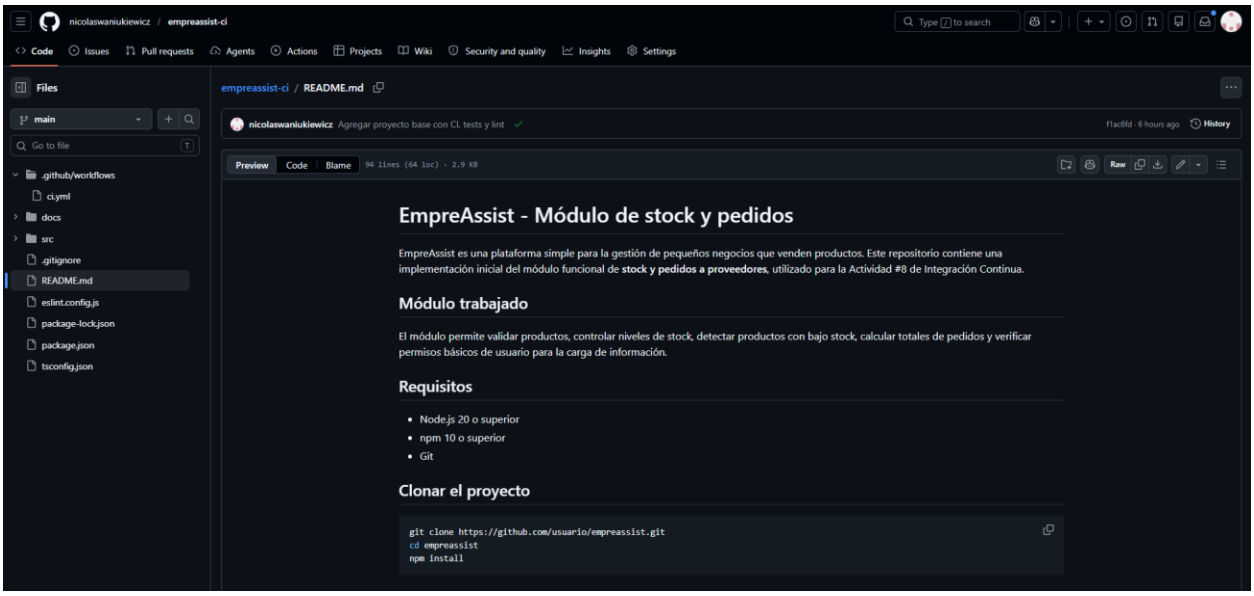

Figura 13. Archivo README.md del repositorio con instrucciones de instalación, ejecución y explicación del proceso de integración continua.

15. Recomendaciones y aprendizajes

- No trabajar directamente sobre main; usar ramas por integrante y pull requests.
- Mantener pruebas unitarias sobre reglas de negocio críticas, especialmente stock, permisos y cálculos.
- Configurar el pipeline desde el inicio del proyecto para detectar errores tempranos.
- Revisar el estado del pipeline antes de fusionar ramas.
- Actualizar el README cada vez que cambien los comandos o la estructura del proyecto.

- Guardar capturas de fallos y correcciones como evidencia académica.

16. Conclusión

La integración continua aplicada al módulo de stock y pedidos de EmpreAssist permite mejorar el control del proceso de desarrollo, reducir errores y asegurar que cada cambio cumpla condiciones mínimas antes de integrarse. La configuración de Git, ramas por integrante, pull requests, pruebas unitarias, linter y build automatizado representa una aplicación concreta de la Gestión de la Configuración, ya que permite identificar elementos, controlar versiones, verificar cambios y conservar evidencias. En términos del proyecto TIF, esta práctica favorece la calidad del producto y la organización del equipo, especialmente porque EmpreAssist busca ser una herramienta confiable para usuarios que necesitan orden, simplicidad y reducción de errores en su negocio.

Referencias

GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions

OpenJS Foundation. (s. f.). ESLint documentation. https://eslint.org/docs/latest/

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson.

Vitest. (s. f.). Vitest documentation. https://vitest.dev/